Практическая работа №5

Интеграция Flask и SQLite (добавление данных)

Герасимов Кирилл Андреевич 3-ИСП44-11

Ссылка на репозиторий: https://github.com/gerasimovkirill022-jpg/prak

Кратко о работе

Цель: Добавить на страницу HTML-форму для создания задачи, обработать POST-запрос во Flask, выполнить INSERT в базу и применить паттерн Post/Redirect/Get.

Технологии: Python 3; Flask (request.form, redirect, url_for); Jinja2; SQLite; HTML-формы, метод POST; паттерн PRG.

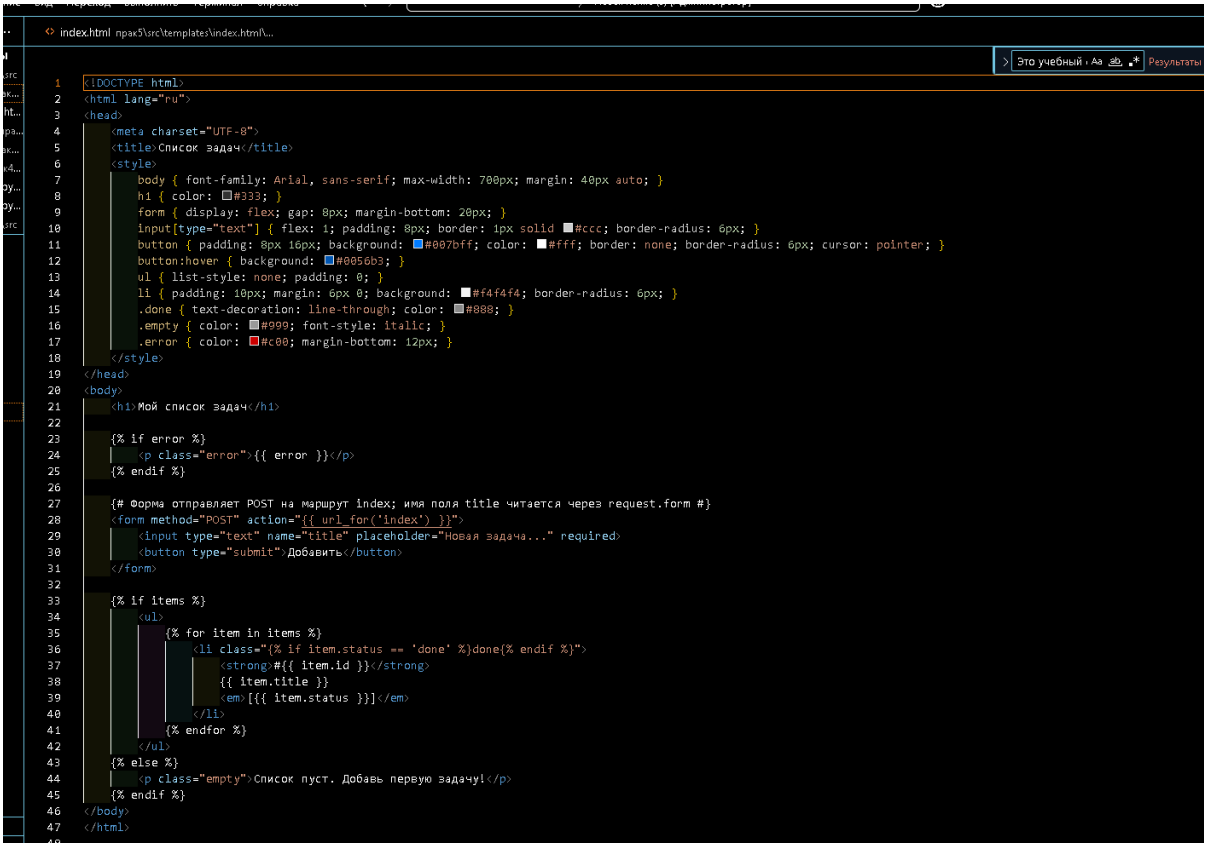
Реализация: Форма отправляет POST на адрес /. Сервер читает request.form.get('title', '').strip(), проверяет, что поле не пустое (серверная валидация дополняет атрибут required), выполняет параметризованный INSERT и commit(), затем отвечает редиректом 302 на GET /. Поэтому обновление страницы (F5) не дублирует задачу.

Основные модули:

- `src/app.py` — маршрут / принимает GET и POST; функция fetch_all_tasks() читает задачи; ветка POST проверяет название, выполняет INSERT, commit и редирект.
- `src/templates/index.html` — добавлена форма <form method="POST"> с полем title и блок вывода ошибки {{ error }}.

Ход работы

Шаг 1. Обновляем шаблон — добавляем форму



```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Список задач</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 700px; margin: 40px auto; }
        h1 { color: #333; }
        form { display: flex; gap: 8px; margin-bottom: 20px; }
        input[type="text"] { flex: 1; padding: 8px; border: 1px solid #ccc; border-radius: 6px; }
        button { padding: 8px 16px; background: #007bff; color: #fff; border: none; border-radius: 6px; cursor: pointer; }
        button:hover { background: #0056b3; }
        ul { list-style: none; padding: 0; }
        li { padding: 10px; margin: 6px 0; background: #f4f4f4; border-radius: 6px; }
        .done { text-decoration: line-through; color: #888; }
        .empty { color: #999; font-style: italic; }
        .error { color: #c00; margin-bottom: 12px; }
    </style>
</head>
<body>
    <h1>Мой список задач</h1>

    {% if error %}
        <p class="error">{{ error }}</p>
    {% endif %}

    {# Форма отправляет POST на маршрут index; имя поля title читается через request.form #}
    <form method="POST" action="{{ url_for('index') }}">
        <input type="text" name="title" placeholder="Новая задача..." required>
        <button type="submit">Добавить</button>
    </form>

    {% if items %}
        <ul>
            {% for item in items %}
                <li class="{% if item.status == 'done' %}done{% endif %}">
                    <strong>#{{ item.id }}</strong>
                    {{ item.title }}
                    <em>[{{ item.status }}]</em>
                </li>
            {% endfor %}
        </ul>
    {% else %}
        <p class="empty">Список пуст. Добавь первую задачу!</p>
    {% endif %}
</body>
</html>
```

Рисунок 1 — шаблон index.html: форма method="POST", поле title, вывод ошибки {{ error }}

Шаг 2. Обновляем app.py: обработка POST и INSERT



```
import sqlite3
from flask import Flask, render_template, request, redirect, url_for
app = Flask(__name__)
DB_PATH = 'app.db'
def get_connection():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn
def fetch_all_tasks():
    """Возвращает все задачи. Нужна и для GET, и для показа ошибки при POST."""
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT id, title, status FROM tasks ORDER BY id")
    rows = cursor.fetchall()
    conn.close()
    return rows
@app.route('/', methods=['GET', 'POST'])
def index():
    # ---- POST: пришла форма ----
    if request.method == 'POST':
        title = request.form.get('title', '').strip()

        # Серверная валидация (атрибут required в браузере можно обойти)
        if not title:
            return render_template(
                'index.html',
                items=fetch_all_tasks(),
                error='Название задачи не может быть пустым.'
            )

        conn = get_connection()
        cursor = conn.cursor()
        cursor.execute("INSERT INTO tasks (title) VALUES (?)", (title,))
        conn.commit()
        conn.close()

        # PRG: после POST — редирект на GET, чтобы F5 не дублировал задачу
        return redirect(url_for('index'))

    # ---- GET: показать страницу ----
    return render_template('index.html', items=fetch_all_tasks())

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

Рисунок 2 — app.py: маршрут с methods=['GET', 'POST'], проверка названия, INSERT и redirect

Шаг 3. Запуск и проверка



Рисунок 3 — страница с формой добавления задачи, база пуста

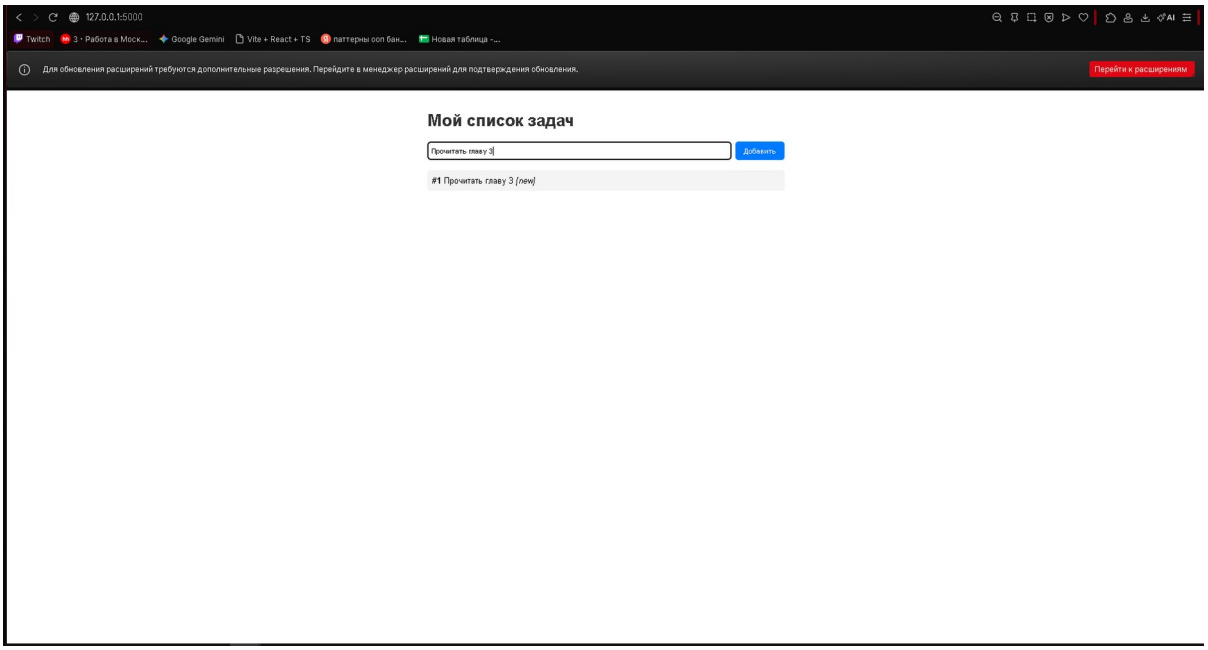


Рисунок 4 — задача «Прочитать главу 3» добавлена через форму и появилась в списке

Шаг 5. Эксперимент с SQL-инъекцией

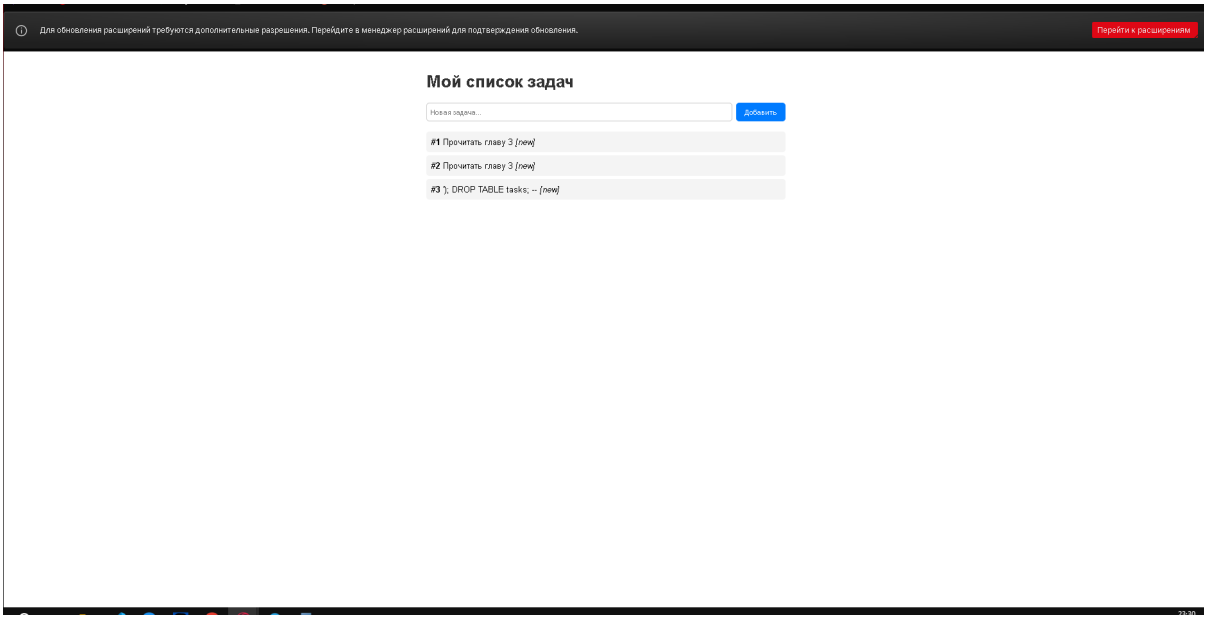


Рисунок 5 — задача «'); DROP TABLE tasks; --» сохранилась как обычный текст, таблица цела

Шаг 6. Проверка кода ответа redirect

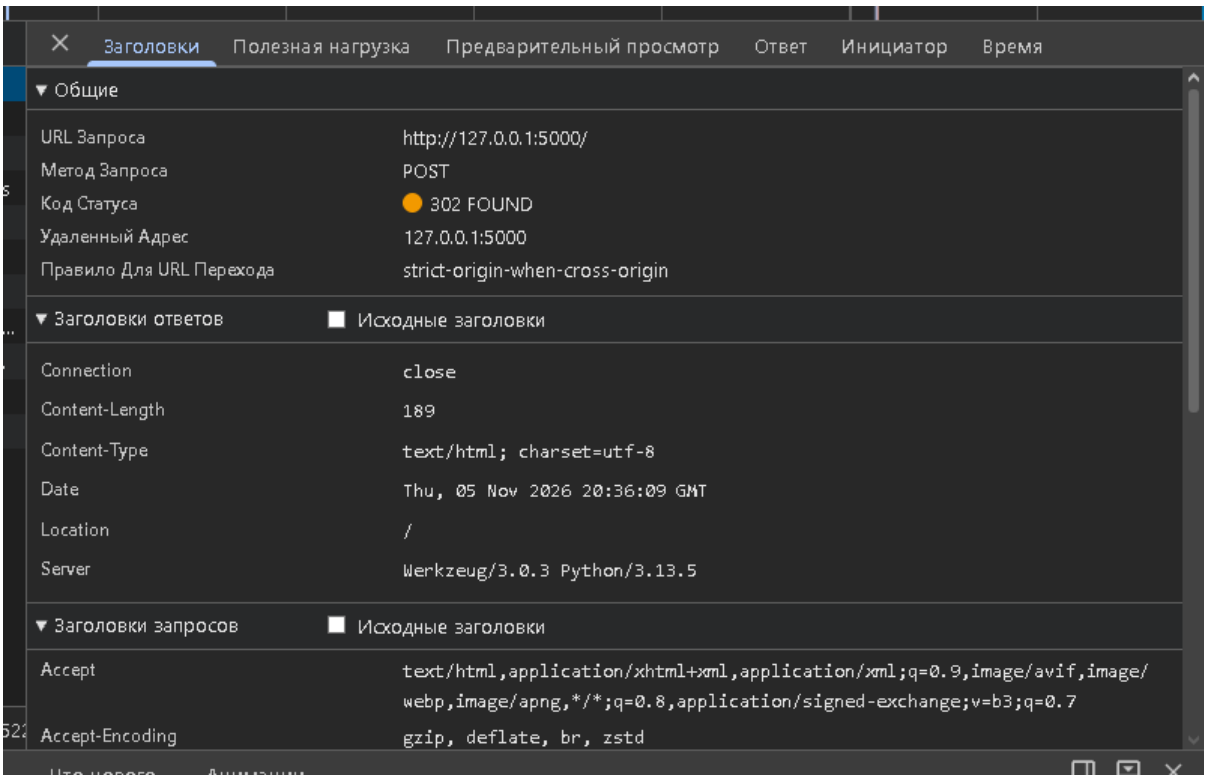


Рисунок 6 — DevTools → Сеть: отправка формы — POST / , ответ 302 FOUND, Location: /

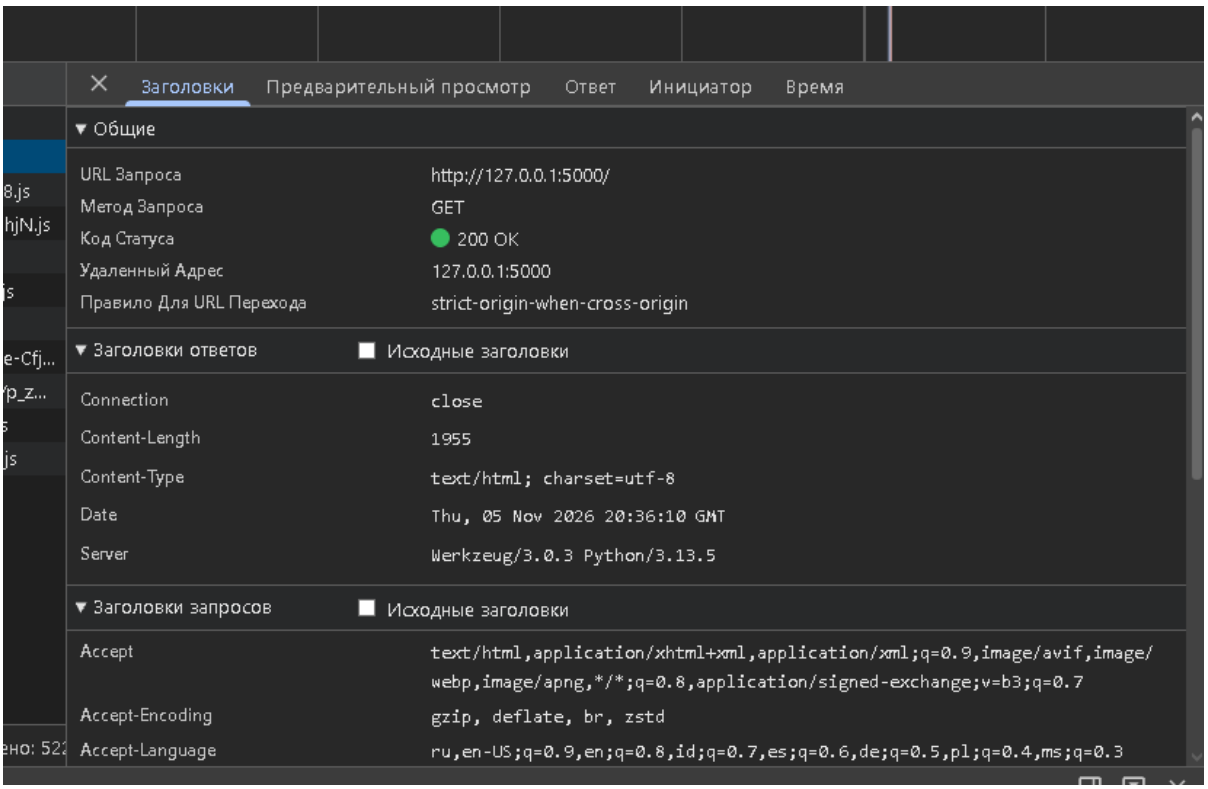


Рисунок 7 — браузер перешёл по редиректу — GET / , ответ 200 OK (паттерн Post/Redirect/Get)

Примеры запросов и ответов

```
POST /  title=Прочитать главу 3  → 302 Found, Location: /
GET /                            → 200 OK (задача в списке)
POST /  title=(пусто)            → 200 OK, «Название задачи не может быть пустым.»
```

Ответы на контрольные вопросы

1. **Чем POST отличается от GET и почему формы отправляют POST?**

GET получает данные, параметры видны в URL, запрос можно безопасно повторять. POST отправляет данные в теле запроса и меняет состояние сервера. Форма создаёт новую запись, поэтому используется POST.

2. **Что будет, если убрать methods=['GET', 'POST'] из маршрута?**

Маршрут будет принимать только GET, и отправка формы вернёт 405 Method Not Allowed.

3. **Зачем нужен паттерн Post/Redirect/Get?**

После POST сервер делает редирект на GET-страницу. Без него F5 повторит POST и задача добавится ещё раз.

4. **Почему url_for('index') лучше жёсткой строки '/'?**

url_for строит адрес по имени функции, поэтому при изменении URL маршрута ссылки и редиректы не сломаются.

5. **Почему request.form.get('title', '') безопаснее request.form['title']?**

Если поля нет, get вернёт пустую строку, а обращение по ключу вызовет ошибку 400 Bad Request (BadRequestKeyError).

6. **Зачем серверная валидация, если есть required?**

required проверяет только браузер; его можно удалить в DevTools или отправить запрос в обход формы. Сервер обязан проверять данные сам.

7. **В каком порядке идут операции при добавлении задачи?**

Подключение → INSERT с параметром → commit() → закрытие соединения → редирект 302 → браузер делает GET / → SELECT и вывод списка.

Вывод

Реализовано добавление задач через веб-форму с серверной валидацией и паттерном PRG. Проверено, что задача сохраняется в базе, не дублируется при обновлении и что SQL-инъекция сохраняется как текст.